

智能化软件工程与多Agent协同开发平台

课题实验与科研推进计划说明书

项目导言

本项目旨在通过构建"向导Agent"与"编程Agent"的双脑协同平台，探索非专业人员通过自然语言驱动软件生成的最佳范式。为确保参与本课题的大一本科生能够平滑跨越认知壁垒，真正掌握Agent编程（Agent-based Programming）的核心机理并具备学术科研能力，特制定本四阶段实验计划。

第一阶段：AI 直觉建立与物理基建准备

预计用时：1周

阶段目标：消除学生对大模型的"神秘感"与"黑盒恐惧"，完成从"普通聊天用户"到"API 调用者"再到"Agent 指挥官"的身份转换，并在本地电脑上搭建好一套专业级的 Agent 开发与测试工作台。

1. 大模型对话的基本原理（认知破冰）

- 文字接龙原理（Next-Token Prediction）：**用最通俗的语言（不需要懂深度学习数学推演），让同学们明白大模型本质上是一个基于概率的高维"文字接龙"概率机，它没有情绪，只有数学向量——借此打破"机器生灵"的错觉。
- 无记忆特性（Statelessness）：**重点讲解为什么 AI 每次开启新对话都是"重新做人"。让学生深刻理解"上下文（Context）"到底是什么，为什么我们需要把之前的聊天记录打包一次次发给它。
- Prompt Engineering（提示词工程）101：**抛弃废话，学习结构化的提示词构建法——如何设定清晰的角色（Role）、提供背景（Background）、约束输出格式（Format），以及如何通过提供例子（Few-shot）来显著提升 AI 的准确率。

2. 算力货币化：API 的购买、配置与安全（硬核基建）

- **API Key 是什么：** 讲解 API Key 的本质——它是调用云端算力的"数字钱包"和"通行证"。理解 Token 的概念（输入 Token 与输出 Token 的计费差异），学会如何在后台观察自己的消耗，建立"算力成本"意识。
- **实操购买与获取：**
 - 指导学生注册国内外顶尖模型平台，获取主力测试模型（如 MiniMax 2.5、DeepSeek 等国内 SOTA 模型）的 API Key。
 - 针对 Claude 等海外模型，指导使用合规的 API 聚合平台（API Hub / 中转平台）获取调用凭证。
- **安全红线：** 强制进行一次安全教育。教授什么是环境变量（Environment Variables，如 `.env` 文件），严禁将包含 API Key 的代码直接 Push 到 GitHub 等公共仓库——一旦泄露，几分钟内数字钱包就会被黑客刷爆。

3. 多脑聚合枢纽：Cherry Studio 的配置与使用

- **为什么不用网页版：** 讲解网页版 ChatGPT/Claude 的局限性（无法自定义底层参数、无法多模型对比、对话容易丢失）。
- **安装与实操：** 指导学生在本地安装 Cherry Studio（或其他类似的本地 Agent 客户端），将上一步获取的各家 API Key 填入客户端，跑通网络请求，完成第一个自定义助手的对话。
- **高阶玩法体验：** 学会如何在客户端内设定复杂的"系统提示词（System Prompt）"，调整 Temperature（温度值，控制 AI 的创造力/严谨度），并在同一个界面下，用相同的问题对比不同模型的回答质量。

4. 终端里的幽灵：Claude Code 的安装与初探

- **从 Chatbot 到 Agent 的跨越：** 讲解 Cherry Studio 主要是"聊天与规划"，而我们要做的系统还需要"手和脚"——这就是 Claude Code 这类终端 Agent 的意义。它不仅能读取你的本地文件夹，还能自动写代码并运行终端命令。
- **环境依赖准备：** 带领新生安装 Node.js (npm) 和 Git 等基础程序员工具。
- **安装 Claude Code：**
 - 在命令行执行安装指令（如 `npm install -g @anthropic-ai/claude-code`）。
 - 完成终端内的 API 授权与鉴权绑定。

- **First Blood (初次连通)**： 在一个空文件夹内唤醒 Claude Code，下达第一条指令（例如："查看当前目录下有什么文件，并帮我新建一个写有 Hello World 的 Python 脚本"），亲眼见证 Agent 是如何自主思考、请求权限并执行系统命令的。

5. 建立人机协作的"语感" (本周小作业)

- **日常任务 AI 化**： 强制要求在这一周内，同学们必须利用配好的 Cherry Studio 或 Claude Code，用纯自然语言让 AI 帮你完成至少两项日常任务（例如：梳理一份本学期的课表时间线、或者让 AI 帮忙精简一段社团活动的汇报文案）。通过实际对话，体会"指令不够清晰时，AI 是如何胡说八道的"，借此培养用自然语言清晰表述意图的"语感"。

里程碑节点：第一阶段组会

项目	内容
会议主题	《数字工作台点亮与 AI "水感"初体验》
会议形式	实操 Show & 答疑会。

会议内容：

- 确保每一位同学的 API 均已跑通，Claude Code 均能在终端唤醒，不让任何一个人在环境配置阶段掉队。
- 分享本周在 Cherry Studio 里测试 Prompt 的心得：AI 最容易在什么地方产生"幻觉"？
- 讨论：Claude Code 这种能直接控制你电脑的 Agent，如果失控了会有什么危险？我们需要什么样的"护栏"？

通过本次组会，确保全员环境就绪，同时让大一新生们用客观、理性的工程师视角去看待大模型——既不盲目崇拜，也不因噎废食，为第二阶段的"极限踩坑"做好完美的心理与知识铺垫。

第二阶段：极限踩坑与直觉建立

预计用时：1周

阶段目标： 抛弃纯理论，通过高强度的"实战踩坑"，亲身体验 Agent 编程中的能力边界与失控瞬间，培养对大模型高维空间的"水感"。

1. 核心操作技巧探索（提供内部参考指南）

- **如何使用与定义 Skill（工具调用）：** 让大模型不仅能说话，还能操作文件、搜索网络。
- **如何构建记忆：** 强制 Agent 在多轮任务中维护一份 TODO.md 或 State.json。
- **如何控制上下文：** 在 Token 爆炸前，果断指挥 Agent 进行总结交接与记忆重载。
- **如何进行 Harness 控制：** 当 Agent 陷入死循环报错时，如何通过干预执行流进行强制剪枝与回滚。

2. "由浅入深"的 10 个梯度实战案例（Agent驱动开发验证）

要求学生绝不手写核心业务代码，全程通过给 Claude Code 等 Agent 下达指令来完成以下十个软件制品的开发。

Level 1-3（基础交互与文件 I/O）

案例	内容	关键能力目标
案例 1	Hello World 进阶版（带参数解析的 CLI 问候脚本）	Agent 基础代码生成
案例 2	Markdown 批量格式化工具	体会 Agent 对本地文件系统的读取与修改
案例 3	命令行待办事项工具	理解 Agent 如何处理本地 JSON 数据的增删改查

Level 4-6（网络请求与多文件工程）

案例	内容	关键能力目标
案例 4	极简网页数据爬虫	体会 Agent 处理网络请求与 DOM 树解析的能力
案例 5	PyQt/Tkinter 桌面时钟	体验 Agent 生成本地 GUI 界面组件的闭环
案例 6	带有路由机制的简易 RESTful API 后端	体验跨文件、多模块的工程生成

Level 7-10（复杂上下文与 Harness 控制考验）

案例	内容	关键能力目标
案例 7	交互式个人博客站点	前端 HTML/JS/CSS 与后端数据的综合生成
案例 8	外部第三方 API 整合看板	调用天气或股票 API 并做图表可视化，考验 Agent 对陌生 API 文档的阅读理解
案例 9	屎山代码重构与 Debug	给一段故意写错的复杂逻辑，考验指挥 Agent 逆向阅读和调试代码的能力
案例 10	"套娃"工具	写一个极简的 Python 脚本，该脚本能调用大模型 API 去自动重命名指定文件夹内的图片（初探用代码控制大模型的 Meta-programming）

里程碑节点：第二阶段组会

项目	内容
会议主题	《非专业人员的叹息之壁：Agent 编程痛点大起底》
会议形式	踩坑总结会。同学们作为"准专业人员"，汇报在完成 10 个案例中遇到的困难、AI 幻觉、死循环等问题。在此基础上，反向推导：如果是一个毫无编程基础的人来用，系统需要提供什么样的"向导规约"？由此统一对接下来 MVP 开发路线的设想。

第三阶段：最小可行性产品（MVP）攻坚

预计用时：2周

阶段目标： 将发散的认知收拢为工程实践，分组完成双Agent协同大体框架的搭建。

1. 结对编程与路线发散

- 将学生两两分组（Pair Programming）。基于前两周的踩坑经验，各组自由选择技术栈与实现路径。
- **目标界定：** 本阶段不要求实现复杂的数据埋点和后台统计，只要求跑通核心主轴——构建一个"向导 Agent"（负责用人话与用户聊需求），并将需求转化为工程指令，传递给"编程 Agent"（负责生成代码）。

2. 开源项目解构与参考

- 分析最新泄露的 Claude Code 源代码与架构解析文章。
- 研究 GitHub 上如 Prema 等基于 Claude/OpenAI API 实现的编程 Agent 包装器（Wrappers），学习其底层 API 调度的 Harness 构建方法。

里程碑节点：第三阶段总结会

项目	内容
会议主题	《从工程实践到科学实证——科研的大门》
会议形式	各小组 MVP 成果 Demo 演示。
特别环节	主导老师胡老师主讲：针对前一个月的工程探索，进行学术视角的升华。讲解计算机与软件工程领域的学术研究方法论，说明"做系统"与"发论文"的区别，详细介绍后续研究数据应当如何设计、清洗与分析，为下一阶段的正式科研开题奠定基调。

第四阶段：工程重构与科学实证开启

预计用时：1~2个月

阶段目标：告别前期的"草莽探索"，基于 MVP 阶段的经验教训进行正规的软件工程重构；随后正式转入科研轨道，引入非专业受试者进行对照实验，提炼科研成果。

1. 经验复盘与底层架构重构 (Engineering Refactoring)

- 推翻与重建：** 前期的 MVP 代码主要用于"踩坑"和验证 API 连通性，本阶段要求同学们基于正规的软件工程方法论，将系统推翻重写。
- 架构规范化：** 建立前后端分离的标准架构，规范化向导 Agent 与编程 Agent 之间的通信协议与数据结构，确保系统具备高稳定性和错误恢复能力（Harness 容错设计）。
- 数据中台内嵌：** 在重构过程中，全面植入"无侵入式静默埋点"功能。确保系统能自动、结构化地记录用户的每一句原始输入、Agent 的意图翻译过程、LLM 返回的 Token 消耗及代码版本快照。

2. 严谨的对照实验设计 (Controlled Experiment)

- 非专业受试者引入：** 面向商学院、文学院等非计算机专业招募数十名大一/大二学生作为真实的实验受试者。
- 双盲或分组对照：**

- **控制组 (Control Group)**：让受试者使用原生的、无引导的普通大模型对话框（如原版 ChatGPT 或单纯的 Claude 页面）来尝试开发一个特定的软件制品（例如"个人记账网页"）。
- **实验组 (Experimental Group)**：让受试者使用我们重构好的"多 Agent 协同与意图规约平台"，在向导 Agent 的引导下完成同样的开发任务。

3. 多维数据采集与"最佳范式"挖掘

在实验过程中，后台静默收集客观数据：

- **工程效率对比**：MVP 触达时间 (Time-to-MVP)、成功闭环率。
- **资源消耗对比**：单位功能 Token 消耗量、因报错导致的交互冗余率。
- **结合主观反馈 (调查问卷)**：分析非专业用户在两种环境下的"认知摩擦度"与"意图修正频次"。
- **范式提炼 (核心产出)**：通过清洗和比对数据，逆向工程提炼出：非专业人员究竟使用什么样的"自然语言指令结构"，能够最高效地驱动底层大模型生成零Bug的代码？（即寻找 AI 时代的"最佳语言接口范式"）。

4. 总结提炼与论文撰写 (Academic Output)

- **数据可视化**：运用 Python/R 语言等工具对采集到的实证数据进行统计学分析与图表呈现。
- **论文成文**：在指导老师的带领下，按照标准学术论文的结构 (Abstract, Introduction, Methodology, Experiment & Results, Conclusion) 撰写研究报告。
- **投稿筹备**：将研究成果整理，瞄准智能化软件工程 (Intelligent SE) 或人机交互 (HCI) 领域的学术会议/期刊（如 CCF 推荐会议、CSSCI/SSCI 等）进行投稿。

5. 里程碑节点：课题最终结题答辩

项目	内容
会议主题	《让世界听懂人类的意图：我们的实证与答案》
会议形式	模拟学术会议进行。各组不仅要展示重构后的健壮系统，更要用严密的数据图表，论证我们所发现的"自然语言最佳交互指令范式"。完成从"大一新生"到"青年研究员"的最终蜕变。